HackPlatform

A docs-driven frontend prototype that connects hackathon discovery, team formation, submission, and rankings in one flow.

/hackathons /hackathons/[slug] /camp /rankings

PROTOTYPE FE-ONLY

01 / FLOW

End-to-End Flow

The complete user journey without a backend

1. Discover

Compare schedules, rules, prizes, and status with searchable list and detail views.

2. Form Teams

Browse Camp posts, filter open roles, and create a recruitment post with a local profile.

3. Submit

Save a draft first, then promote it to final submission and reflect it in the hackathon leaderboard.

4. Rank

Check public results from the hackathon leaderboard and the global rankings surface.

Product Surfaces & Runtime Shell

Routes stay thin while shared runtime layers keep behavior consistent

/

Landing

Introduces the service, key actions, and the four main product surfaces.

/hackathons + /hackathons/[slug]

Explore + Deep Dive

Searchable listing, multi-filter browsing, and a detail page with overview, teams, submit, and leaderboard sections.

/camp

Team Formation

Camp keeps recruitment posts in local storage and exposes filtered discovery plus a write flow.

/rankings

Public Results

Global rankings stay read-oriented and refresh from the persisted rankings store without coupling to profile edits.

RUNTIME STACK

ROOT LAYOUT + PROVIDERS

cookie-first locale, theme init script, ThemeProvider, I18nProvider

SHARED APP SHELL

bootstrapStorage, TopHeader, HeaderAuth, ToastProvider

PAGE FEATURES

filters, section TOC, submit draft/final, leaderboard + rankings refresh

Architecture & Data Flow

Docs-driven read paths, storage-backed write paths, and clear runtime boundaries

SOURCE OF TRUTH

Docs + Seed JSON

`docs/service-overview.md`
`docs/system-structure.md`
`docs/requirements/예시자료/`

Product scope, runtime contracts, and public read data begin in docs before they reach the UI.

→

NORMALIZE + PERSIST

lib/data + lib/storage

`recovery · bootstrap · write helpers`

Seed data is normalized for UI use, while localStorage keeps profile, teams, submissions, leaderboards, and rankings consistent.

→

ROUTE + VIEW

app + components

`thin routes · shared shell · design system`

Route files stay thin, while feature screens and design-system primitives assemble the actual product experience.

→

USER EFFECT

Readable, writable prototype

Anyone can read public data, while write flows are gated by a local profile and reflected back into the UI immediately.

Locale and theme are restored from the root layout + providers layer before page content mounts.

Feature Proofs

What works in the prototype?

Refresh-safe storage bootstrap

Public seeds and persisted local entities recover cleanly after reload instead of forcing users to restart their flow.

Cookie-first i18n + metadata sync

Korean and English dictionaries are wired from the root layout, then page-level metadata can resync on client screens.

Theme persistence

The initial theme is restored before paint and the chosen mode stays durable through local storage.

Recoverable UI Patterns

Missing hackathon → EmptyState

The detail screen stays usable and explains the missing resource instead of crashing the whole page.

Storage failure → ErrorState

List-heavy screens explicitly separate loading, empty, and storage-unavailable states for readable recovery.

Scroll-spy + rankings refresh

The detail TOC tracks section position, while rankings can refresh from storage and focus-driven updates.

05 / CONCLUSION

Summary & Exclusions

Scope boundaries for this iteration

Out of Scope

- Real-time chat integration
- Actual backend database (Postgres, etc.)
- Production Auth (OAuth / JWT)
- PII collection in UI forms

Validated Core Product Flow

HackPlatform now proves the public hackathon journey end to end: discovery, team formation, submission, leaderboard reflection, and global rankings, all within a docs-driven frontend prototype.

→ **Ready for validation and next iteration**